

¿Qué es un formulario contenedor (MDI)?

Un formulario contenedor MDI es la ventana principal de una aplicación que sirve como espacio de trabajo para mostrar diferentes formularios secundarios o ventanas internas.

MDI significa Multiple Document Interface, que en español puede traducirse como Interfaz de Múltiples Documentos.

En una aplicación MDI, el usuario normalmente tiene una ventana principal y dentro de ella puede abrir diferentes formularios. Estos formularios pueden moverse, cambiar de tamaño, minimizarse o cerrarse sin necesidad de cerrar la ventana principal.

Por ejemplo, en un sistema de ventas podría existir un formulario principal llamado FrmPrincipal que contenga un menú con opciones como:

Clientes

Productos

Ventas

Compras

Reportes

Usuarios

Salir

Al seleccionar "Clientes", se abre un formulario hijo dentro de la ventana principal.

Una estructura sencilla sería:

El diagrama muestra una ventana principal titulada "Sistema de Ventas". Debajo del título hay una barra de menú con cuatro botones: "Archivo", "Mantenimiento", "Reportes" y "Ayuda". En el centro de la ventana principal se encuentra un formulario hijo titulado "Formulario Clientes". Este formulario contiene dos campos de texto etiquetados como "Nombre:" y "Dirección:", cada uno seguido de una línea subrayada para la entrada de datos.

¿Qué es un JDesktopPane?

JDesktopPane es un componente de Java Swing que funciona como un escritorio virtual o área contenedora para los formularios internos de una aplicación MDI.

La documentación oficial de Java indica que JDesktopPane es un contenedor utilizado para crear una interfaz de múltiples documentos o un escritorio virtual. Los objetos JInternalFrame se crean y posteriormente se agregan al JDesktopPane.

Su función principal es permitir que diferentes JInternalFrame puedan existir dentro de una misma ventana principal.

Por ejemplo:

```
JDesktopPane escritorio = new JDesktopPane();  
JInternalFrame formulario = new JInternalFrame(  
    "Clientes",  
    true,  
    true,  
    true,  
    true  
);
```

```
escritorio.add(formulario);
```

```
formulario.setVisible(true);
```

En este ejemplo:

JDesktopPane representa el escritorio.

JInternalFrame representa el formulario hijo.

add() agrega el formulario al escritorio.

setVisible(true) hace visible el formulario.

¿Qué es un JInternalFrame?

JInternalFrame es un componente de Java Swing que representa una ventana interna dentro de un JDesktopPane.

Puede considerarse como un formulario hijo que se encuentra dentro de la ventana principal.

La clase JInternalFrame proporciona características similares a una ventana tradicional, como:

Título.

Movimiento.

Cambio de tamaño.

Cierre.

Minimización.

Maximización.

Barra de menú.

Contenido propio.

La documentación oficial de Java describe JInternalFrame como un objeto ligero que proporciona muchas características de un marco o ventana nativa y señala que normalmente se agregan estos componentes a un JDesktopPane.

Un ejemplo básico sería:

```
JInternalFrame ventana = new JInternalFrame(  
"Formulario de Clientes",  
true,  
true,  
true,  
true
```

```
);
```

```
ventana.setSize(500, 400);
```

```
ventana.setVisible(true);
```

```
desktopPane.add(ventana);
```

Los parámetros utilizados en el constructor indican:

"Formulario de Clientes" → Título

true → Permite cambiar tamaño

true → Permite cerrar

true → Permite maximizar

true → Permite minimizar

Por ejemplo:

```
new JInternalFrame(
```

```
"Clientes",
```

```
true,
```

```
true,
```

```
true,
```

```
true
```

```
);
```

crea una ventana interna llamada Clientes con todas esas funciones habilitadas.

¿Cómo funciona un JMenuBar?

JMenuBar es el componente de Swing que permite crear la barra de menús de una aplicación.

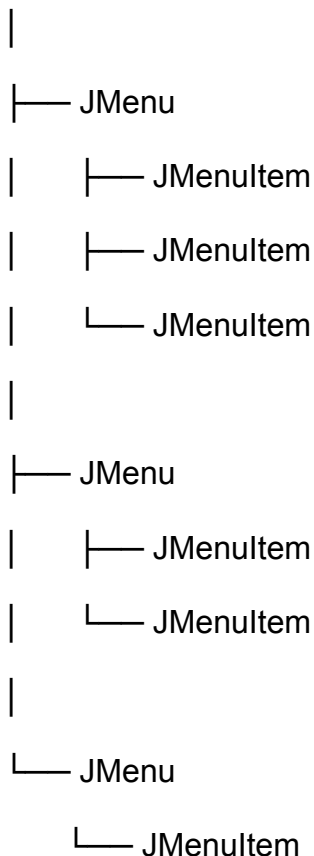
Una barra de menú normalmente se encuentra en la parte superior de la ventana principal y contiene diferentes menús, como:

Archivo Mantenimiento Reportes Ayuda

La documentación oficial indica que JMenuBar es un contenedor para mostrar menús. Dentro de él se agregan objetos JMenu, y estos pueden contener diferentes JMenuItem.

La estructura normalmente es:

JMenuBar



¿Cómo abrir formularios hijos desde un menú?

Para abrir formularios hijos desde un menú se utiliza normalmente un ActionListener.

El proceso básico es:

1. El usuario selecciona una opción del menú.
2. Se ejecuta un ActionListener.
3. Se crea un JInternalFrame.
4. Se agrega el formulario al JDesktopPane.
5. Se establece su posición y tamaño.
6. Se hace visible.

Por ejemplo:

```
private void abrirClientes() {  
  
    JInternalFrame formulario = new JInternalFrame(  
        "Clientes",  
        true,  
        true,  
        true,  
        true  
    );  
  
    formulario.setSize(500, 400);  
    formulario.setLocation(50, 50);  
    desktopPane.add(formulario);  
    formulario.setVisible(true);  
}
```

Después se puede llamar este método desde una opción del menú:

```
private void menuClientesActionPerformed(  

```

```
java.awt.event.ActionEvent evt) {  
    abrirClientes();  
}
```

De esta manera, cuando el usuario haga clic en:

Mantenimiento



Clientes

se abrirá el formulario de clientes dentro del JDesktopPane.

Ventajas de utilizar esta arquitectura en aplicaciones de escritorio

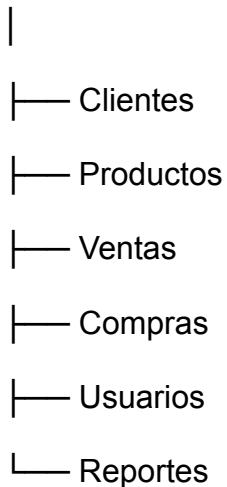
La arquitectura MDI presenta diferentes ventajas.

Organización

Permite organizar diferentes módulos de una aplicación dentro de una ventana principal.

Por ejemplo:

Sistema Administrativo



Esto facilita la navegación del usuario.

Mejor aprovechamiento del espacio

Los diferentes formularios pueden abrirse dentro de una misma ventana y pueden moverse, cambiar de tamaño, minimizarse o maximizarse.

Navegación centralizada

El usuario puede acceder a los diferentes módulos desde la barra de menú sin tener que abrir muchas ventanas independientes del sistema operativo.

Separación de módulos

Cada JInternalFrame puede representar un módulo diferente.

Por ejemplo:

FrmPrincipal

|

├── FrmClientes

├── FrmProductos

├── FrmVentas

└── FrmReportes

Esto ayuda a mantener el proyecto organizado.

Facilidad de mantenimiento

Al separar cada módulo en una clase diferente, resulta más sencillo modificar o corregir una parte de la aplicación sin tener que modificar todo el programa.

Bibliografía / Fuentes consultadas

Oracle. (s. f.). *JDesktopPane Class — Java Platform, Standard Edition API*. Oracle Java Documentation. [Documentación oficial de JDesktopPane](#)

Oracle. (s. f.). *JInternalFrame Class — Java Platform, Standard Edition API*. Oracle Java Documentation. [Documentación oficial de JInternalFrame](#)

Oracle. (s. f.). *JMenuBar Class — Java Platform, Standard Edition API*. Oracle Java Documentation. [Documentación oficial de JMenuBar](#)

Oracle. (s. f.). *The Java Tutorials — How to Use Internal Frames*. Oracle. [Java Tutorials de Oracle sobre Internal Frames](#)